K8S-12: Specialized Monitoring Scenarios

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 12 of 14 | **Created:** January 2026 | **Last Updated:** 10/02/2026

NGINX Ingress, Code-Module Delivery, Resource Tuning, and StatsD Ingestion

This notebook covers specialized monitoring scenarios including NGINX Ingress Controller instrumentation, how code modules reach pods (image volumes, CSI driver, ephemeral volumes), resource sizing, and StatsD metric ingestion on Kubernetes.

Table of Contents

1. NGINX Ingress Controller Monitoring
2. CSI Driver Architecture
3. CSI Driver Resource Configuration
4. Component Sizing Guidelines
5. Telemetry Ingest Configuration
6. StatsD Metrics Ingestion on Kubernetes
7. Troubleshooting Specialized Scenarios

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring
Kubernetes Cluster	Dynatrace Operator v1.0+ installed
NGINX Ingress	ingress-nginx controller (for Section 1)
Knowledge	Completed K8S-01 through K8S-02

1. NGINX Ingress Controller Monitoring

Why Monitor NGINX Ingress?

The NGINX Ingress Controller is often the entry point for all external traffic. Instrumenting it provides:

Benefit	Description
End-to-end traces	Complete request path visibility
Ingress latency	Measure time spent in ingress layer
Error correlation	Link 5xx errors to backend services
Traffic analysis	Understand traffic patterns

Prerequisites for NGINX Instrumentation

Requirement	Details
OneAgent version	1.227+
Pod naming	Pod or container name must contain `ingress-nginx-` or `nginx-ingress-`
Architecture	*"ARM64 architecture is not supported."*

Requirement	Details
Controller type	The official Kubernetes ingress-nginx controller. The F5 NGINX ingress controller is instrumented automatically; derivatives such as Bitnami's are not supported by these steps

Configuration Steps

Step 1: Edit the ingress-nginx-controller ConfigMap

```
kubectl edit configmap ingress-nginx-controller -n ingress-nginx
```

Step 2: Add the OneAgent module loading snippet (path for `cloudNativeFullStack` and `applicationMonitoring`; `classicFullStack` uses `/opt/dynatrace/oneagent/...` instead of `/opt/dynatrace/oneagent-paas/...`)

```
data:
  main-snippet: |
    load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-x86-64/liboneagentnginx.so;
```

Step 3 (Optional): Add trace context to access logs

```
data:
  main-snippet: |
    load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-x86-64/liboneagentnginx.so;
  log-format-upstream: &gt;-
    $remote_addr - $remote_user [$time_local] "$request"
    [!dt dt.trace_id=$dt_trace_id,dt.span_id=$dt_span_id,dt.trace_sampled=$dt_trace_sampled]
    $status $body_bytes_sent "$http_referer" "$http_user_agent"
```

Step 4: Rolling restart the ingress controller

```
kubectl rollout restart deployment ingress-nginx-controller -n ingress-nginx

# Watch rollout
kubectl rollout status deployment ingress-nginx-controller -n ingress-nginx
```

Verifying NGINX Instrumentation

```
# Check OneAgent injection
kubectl -n ingress-nginx exec -it deploy/ingress-nginx-controller -- cat
/proc/1/maps | grep dynatrace

# Check for errors in logs
kubectl -n ingress-nginx logs -l app.kubernetes.io/name=ingress-nginx | grep -
i dynatrace
```

> **Note:** No DynaKube changes are required for NGINX monitoring - only the ConfigMap edit and pod restart.
>
> **Sources:** Instrument ingress-nginx (DT docs) — *"The pod or container name must contain the substring ingress-nginx- or nginx-ingress- to ensure proper instrumentation of the NGINX binary."*

```
// Check if NGINX Ingress spans are arriving
// dt.service.name, not service.name: service.name is the OpenTelemetry
resource attribute and is
// absent on OneAgent spans (on the validation tenant, 10/02/2026, it was set
on 4,258 of 304,602
// spans carrying k8s.namespace.name). dt.service.name is set on every span.
fetch spans, from:-1h
| filter contains(dt.service.name, "nginx") or contains(dt.service.name,
"ingress")
| summarize count = count(), by:{dt.service.name, span.name}
| sort count desc
| limit 20
```

2. CSI Driver Architecture

Understanding the CSI Driver

The CSI (Container Storage Interface) Driver provides OneAgent code modules to application pods via volumes instead of host mounts.

Code-Module Delivery Is a Choice

There are three ways to get OneAgent code modules into application pods. Operator 1.10.0 (released 07/15/2026) added a CSI-to-ephemeral-volume migration mode, and **Operator 1.11.0 (released 10/01/2026) adds image volumes**, which the release notes call *"a more secure, storage-efficient, and reliable way to instrument application pods that replaces the CSI driver as the recommended approach."*

	Image volumes (Operator 1.11.0+)	**CSI driver**	**Ephemeral volumes (no CSI driver)**
How modules reach the pod	Each node pulls the code-modules image once and shares that copy with every instrumented pod	Cached once per node by the CSI DaemonSet, then mounted into each injected pod	Provisioned into each pod's own ephemeral volume
Cluster footprint	No DaemonSet, no elevated privileges	A 5-container privileged DaemonSet per node, with a host-path CSI socket	No DaemonSet; work moves into the injection path
Requirements	Kubernetes 1.35+, containerd 2.2+ or CRI-O 1.33+; not compatible with the built-in tenant registry	Any supported cluster; Helm default, or the `kubernetes-csi.yaml` manifest	Any supported cluster; `csidriver.enabled: false`, or the plain `kubernetes.yaml` manifest
Density behavior	One copy per node	One cached copy per node	One copy per pod — *"storage-inefficient"* per the docs: 1 GB of ephemeral storage per monitored pod, against 0.1 GB per injected pod with the CSI driver

	Image volumes (Operator 1.11.0+)	CSI driver	Ephemeral volumes (no CSI driver)
Choose it when	Operator 1.11.0+ and the node requirements are met — Dynatrace's recommended approach	The cluster cannot meet the image-volume requirements and can run a privileged DaemonSet	Neither: a CSI DaemonSet is unacceptable (see FAQ-13 for OpenShift SCCs) and image volumes are unavailable

What you get without choosing. It depends on how you install. The Helm chart turns the CSI driver on (`csidriver.enabled: true` in the 1.9.0, 1.10.2 and 1.11.0 charts). With manifests it depends on the file: every release from 1.0.0 to 1.11.0 ships both `kubernetes.yaml`, which contains no CSI driver, and `kubernetes-csi.yaml`, which adds it. So running without the CSI driver is not new in 1.10.0; what 1.10.0 added is the migration mode below. Without the CSI driver, injected pods get their code modules through an ephemeral volume. The image-volume guide calls that *"Node Image Pull via ephemeral volume injection, which is the default since Dynatrace Operator version 1.10"*, and the 1.10.0 notes say *"CSI-less codeModulesImage injection does not require this flag"* (`feature.dynatrace.com/node-image-pull`). Read that as the default *for CSI-less injection*, not as ephemeral volumes becoming the Operator's overall default: a Helm install still starts on CSI.

> **Sources:**
>
> - Operator Helm chart values, v1.11.0 (Dynatrace GitHub) — `csidriver.enabled: true` ; the same default read at v1.9.0 and v1.10.2, 10/02/2026
> - Operator releases (Dynatrace GitHub) — `kubernetes.yaml` and `kubernetes-csi.yaml` assets at v1.0.0, v1.9.0, v1.10.0 and v1.11.0; the CSI DaemonSet is present only in the `-csi` file (read 10/02/2026)
> - Operator 1.10.0 release notes (DT docs) — *"The feature.dynatrace.com/node-image-pull feature flag now only affects the CSI driver."*
> - Use image volumes for code modules injection (DT docs) — *"ephemeral volume injection needs no privileged access but is storage-inefficient"*
> - Storage requirements (DT docs) — `applicationMonitoring` , CSI driver disabled: *"1 GB per monitored pod from local ephemeral storage"*; enabled: 0.1 GB per injected pod plus 1 GB per tenant and OneAgent version on the node

> ˜ **Derived:** "not the overall default" combines the chart default with the guide's sentence, which describes what pods revert to *"If the CSI driver is disabled"*

Decision guidance: on Operator 1.11.0+ with Kubernetes 1.35+ and a supported runtime, move to image volumes. Otherwise stay on the CSI driver unless platform policy rules out a privileged DaemonSet, in which case use ephemeral volumes.

Turning image volumes on. Set the DynaKube feature flag `feature.dynatrace.com/mount-code-modules-via-image-volume: "true"` (it cannot be combined with `node-image-pull`). Existing pods keep their current mounts until restarted, so a full switch needs a rolling restart of instrumented workloads. To try it on one workload first, annotate that pod `oneagent.dynatrace.com/volume-type: "image"`.

> **Sources:** Operator 1.11.0 release notes (DT docs) — *"replaces the CSI driver as the recommended approach"*.

Staged-rollout caveat. Image volumes need **Operator 1.11.0** and the one-step CSI → ephemeral migration mode needs **Operator 1.10.0**, and estates upgrade on their own schedule. Running without the CSI driver does not depend on either. What a 1.9.x or older cluster lacks is `csidriver.migrationMode`, so moving an *existing* CSI cluster off the driver is not the single-restart procedure below. The how-to measures migration mode as *"a single pod-restart cycle instead of two"*, so without it plan for two — or upgrade to 1.10.0+ first. Verify your operator version before planning a migration:

```
kubectl -n dynatrace get deployment dynatrace-operator \
  -o jsonpath='{.spec.template.spec.containers[0].image}'
```

Migrating CSI → Ephemeral Volumes (Operator 1.10.0+)

The documented migration direction is **CSI → ephemeral**. The Dynatrace how-to uses the `csidriver.migrationMode` Helm value so the move takes *"a single pod-restart cycle instead of two"*. Prerequisites: *"Dynatrace Operator version 1.10.0+"*, an existing CSI-based injection setup, and a Helm-managed Operator — *"For manifest-based installations, see Migrate from manifests to Helm first."* Application manifests are untouched throughout; the change is Helm values plus one restart cycle.

Step 1 — Enable migration mode. The CSI DaemonSet keeps running (so existing mounts can be cleanly unmounted) while new injections switch to ephemeral volumes:

```
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --namespace dynatrace \
  --reset-then-reuse-values \
  --set csidriver.enabled=true \
  --set csidriver.migrationMode=true \
  --atomic
```

"After this upgrade, all newly injected pods use ephemeral-volume injection. Existing pods continue to use their current CSI mounts until they are restarted."

Step 2 — Restart every injected workload so the webhook re-injects it with ephemeral volumes:

```
kubectl rollout restart deployment &lt;deployment-name&gt; -n
&lt;namespace&gt;
```

Step 3 — Verify no pod still mounts a CSI volume. *"Empty output means all pods have been migrated."* Restart the workload of any pod that is listed:

```
kubectl get pods --all-namespaces -o jsonpath='{range .items[?
(@.spec.volumes[*].csi.driver=="csi.oneagent.dynatrace.com")]}
{.metadata.namespace}{"\t"}{.metadata.name}{"\n"}{end}'
```

Step 4 — Only then disable the CSI driver. *"Any pods that still rely on CSI mounts will fail to function after the CSI driver is disabled."* — never set `csidriver.enabled=false` before Step 3 comes back empty:

```
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --namespace dynatrace \
  --reset-then-reuse-values \
  --set csidriver.enabled=false \
  --atomic
```

Roll it per cluster rather than fleet-wide, and re-check pod-start latency at your busiest node density before committing.

> Source: Migrate from CSI driver to ephemeral volumes (DT docs), read 09/18/2026.

CSI Driver Container Structure

The CSI Driver DaemonSet runs **5 containers** in a sidecar pattern:

Container	Purpose	Resource Impact
`csi-init`	Initialize volume plugin	Runs once at startup
`server`	Main CSI plugin	Core functionality
`provisioner`	Volume provisioning	Highest resource needs
`registrar`	Node registration	Low overhead
`livenessprobe`	Health checking	Minimal

3. CSI Driver Resource Configuration

> **This section applies only on the CSI path.** If you chose image volumes (Operator 1.11.0+) or run without the CSI driver (ephemeral volumes, §2) there is no CSI DaemonSet to size and none of the values below exist — the equivalent work is watching per-pod volume provisioning and pod-start latency instead. That includes clusters installed from the plain `kubernetes.yaml` manifest, which has no CSI driver on any Operator version. On a Helm install the CSI driver is on unless you turned it off, so by default this section applies.

Helm Values for CSI Driver

Configure resources for each CSI Driver container in your Helm `values.yaml`. The values below are suggested starting points with more headroom than the chart defaults (compared in the table that follows):

```yaml
csidriver:
  enabled: true

  # Init container - runs once at startup
  csiInit:
    resources:
      requests:
        cpu: 50m
        memory: 100Mi
      limits:
        cpu: 100m
        memory: 128Mi

  # Main CSI plugin
  server:
    resources:
      requests:
        cpu: 50m
        memory: 100Mi
      limits:
        cpu: 100m
        memory: 128Mi

  # Volume provisioner - needs more resources
  provisioner:
    resources:
      requests:
        cpu: 300m
        memory: 100Mi
      limits:
        cpu: 500m         # Often missing in defaults!
        memory: 256Mi     # Often missing in defaults!

  # Node registration sidecar
  registrar:
    resources:
      requests:
        cpu: 20m
        memory: 30Mi
      limits:
        cpu: 50m
        memory: 64Mi

  # Health check sidecar
  livenessprobe:
    resources:
      requests:
```

```
      cpu: 20m
      memory: 30Mi
    limits:
      cpu: 50m
      memory: 64Mi
```

CSI Driver Resource Summary Table

Container	Chart default (Operator 1.11.0) requests / limits	Suggested above	Notes
`csiInit`	50m, 100Mi / 50m, 100Mi	50m, 100Mi / 100m, 128Mi	Init container
`server`	50m, 100Mi / 50m, 100Mi	50m, 100Mi / 100m, 128Mi	Main plugin
`provisioner`	300m, 100Mi / **none**	300m, 100Mi / 500m, 256Mi	**No limits by default**
`registrar`	20m, 30Mi / 20m, 30Mi	20m, 30Mi / 50m, 64Mi	Sidecar
`livenessprobe`	20m, 30Mi / 20m, 30Mi	20m, 30Mi / 50m, 64Mi	Sidecar

> **Warning:** In the Operator 1.11.0 chart the `provisioner` container has requests but no limits. Add limits if your admission policy requires them. The suggested figures are community practice, not Dynatrace guidance — size from what the containers use on your busiest nodes.
>
> **Sources:** Operator Helm chart values, v1.11.0 (Dynatrace GitHub).

4. Component Sizing Guidelines

These tables are starting points from community practice, not Dynatrace-published figures. Measure each component's real usage (the § 7 restart query and K8S-04's usage-versus-requests queries) and adjust; for ActiveGate capacity planning see FAQ-10.

ActiveGate Sizing

Cluster Size	Nodes	CPU Limit	Memory Limit	Replicas
Small	1-10	500m	1Gi	1
Medium	10-50	1000m	2Gi	2
Large	50-100	2000m	4Gi	2
Enterprise	100+	4000m	8Gi	3+

OneAgent Resources

Mode	CPU Request	CPU Limit	Memory Request	Memory Limit
cloudNativeFullStack	100m	300m	256Mi	512Mi
classicFullStack (legacy)	150m	500m	512Mi	1Gi
applicationMonitoring	50m	200m	128Mi	256Mi

OTel Collector Sizing

Telemetry Volume	CPU Limit	Memory Limit	Use Case
Low	200m	256Mi	Dev/Test
Medium	500m	512Mi	Staging
High	1000m	1Gi	Production
Very High	2000m	2Gi	High-throughput

Log Monitoring Sizing

Log Volume	CPU Limit	Memory Limit	Notes
Low	200m	256Mi	< 1GB/day
Medium	500m	512Mi	1-10GB/day

Log Volume	CPU Limit	Memory Limit	Notes
High	1000m	1Gi	10-50GB/day
Very High	2000m	2Gi	50GB+/day

5. Telemetry Ingest Configuration

Multi-Protocol Ingestion

Configure DynaKube to accept telemetry from multiple sources:

```
spec:
  telemetryIngest:
    protocols:
      - otlp      # OpenTelemetry Protocol
      - jaeger    # Jaeger traces
      - zipkin    # Zipkin traces
      - statsd    # StatsD metrics
    serviceName: telemetry-ingest
```

The Operator deploys the Dynatrace OpenTelemetry Collector behind a Service named `<dynakube-name>-telemetry-ingest` in the `dynatrace` namespace; `serviceName` above renames it to `telemetry-ingest`. The data ingest token needs `openTelemetryTrace.ingest`, `logs.ingest` and `metrics.ingest` (Operator 1.6+).

Protocol Endpoints

Protocol	Port	Endpoint (with `serviceName: telemetry-ingest`)
OTLP gRPC	4317 (TCP)	`telemetry-ingest.dynatrace:4317`
OTLP HTTP	4318 (TCP)	`http://telemetry-ingest.dynatrace:4318`
Zipkin	9411 (TCP)	`http://telemetry-ingest.dynatrace:9411`
Jaeger gRPC	14250 (TCP)	`telemetry-ingest.dynatrace:14250`

Protocol	Port	Endpoint (with `serviceName: telemetry-ingest`)
Jaeger Thrift HTTP	14268 (TCP)	`http://telemetry-ingest.dynatrace:14268`
Jaeger Thrift Compact / Binary	6831 / 6832 (UDP)	`telemetry-ingest.dynatrace:6831`
StatsD	8125 (UDP)	`telemetry-ingest.dynatrace:8125`

> **Sources:** Enable Dynatrace telemetry ingest endpoints (DT docs) — ports reference; *"The data ingest token requires the token scopes openTelemetryTrace.ingest , logs.ingest , and metrics.ingest"*.

Application Configuration

```
# Application deployment using OTLP
env:
  - name: OTEL_EXPORTER_OTLP_ENDPOINT
    value: "http://telemetry-ingest.dynatrace:4318"
  - name: OTEL_SERVICE_NAME
    value: "my-application"
```

```
// CSI driver health — container restarts per CSI pod (last 24 h)
// A provisioner or server container restarting repeatedly is the liveness-probe crash loop
// described in K8S-09 § 2. The restart counter is written only when a restart happened, so an
// empty result means no restarts. Executed 10/02/2026.
timeseries r = sum(dt.kubernetes.container.restarts), from:-24h,
  filter:{k8s.workload.name == "dynatrace-oneagent-csi-driver"},
  by:{k8s.cluster.name, k8s.pod.name, k8s.container.name}
| fieldsAdd restarts = arraySum(r)
| filter restarts &gt; 0
| fields k8s.cluster.name, k8s.pod.name, k8s.container.name, restarts
| sort restarts desc
```

```
// Spans that carry an OpenTelemetry instrumentation scope, by service
// otel.scope.name is set by OpenTelemetry instrumentation; this shows which services send it.
fetch spans, from:-1h
| filter isNotNull(otel.scope.name)
| summarize span_count = count(), by:{dt.service.name, otel.scope.name}
| sort span_count desc
| limit 15
```

6. StatsD Metrics Ingestion on Kubernetes

The Challenge

Many applications emit custom metrics using the StatsD protocol (UDP port 8125). On VMs, Dynatrace OneAgent includes a built-in StatsD daemon, but the docs state: *"OneAgent deployed on Kubernetes, for example using Dynatrace Operator, isn't supported. For Kubernetes environments, we recommend remote StatsD monitoring using an environment ActiveGate."*

Ingestion Approaches for Kubernetes

Approach	Works on K8s?	Notes
Environment ActiveGate as remote listener	Yes	The approach the StatsD docs recommend for Kubernetes
DynaKube `telemetryIngest` with `statsd` (Operator 1.6+)	Yes	The Operator deploys and manages the Dynatrace OTel Collector with a StatsD endpoint on UDP 8125 (§ 5)
Self-managed OpenTelemetry Collector	Yes	Documented by Dynatrace; the pattern below, when you want to own the collector
OneAgent StatsD daemon	No	*"OneAgent deployed on Kubernetes, for example using Dynatrace Operator, isn't supported."*
Telegraf	Community practice	Not documented by Dynatrace

Self-managed: OpenTelemetry Collector with StatsD Receiver

If you run the DynaKube `telemetryIngest` endpoint (§ 5), point StatsD clients at `telemetry-ingest.dynatrace:8125` and skip the rest of this section. To own the collector yourself, deploy a dedicated OTel Collector that listens for StatsD traffic, converts it to OTLP, and ships it to Dynatrace. The collector is **shared infrastructure** — it gets its own Deployment, separate from your application pods.

Deployment Pattern Options

Pattern	When to Use	Notes
Cluster-wide Deployment	Most common starting point	Single Deployment in a `monitoring` namespace. All apps send StatsD via FQDN.
Per-namespace Deployment	Isolated teams/environments	One collector per namespace. More isolation, more overhead.
DaemonSet	Very high metric volume	One pod per node. Avoids cross-node traffic. Overkill for most StatsD use cases.

> **Recommendation:** Start with a cluster-wide Deployment in a dedicated `monitoring` or `observability` namespace. The platform/infra team manages the collector independently from application deployments.

Step-by-Step Setup

Step 1: Create the collector configuration

```
# otelcol-config.yaml (stored as a ConfigMap)
receivers:
  statsd:
    endpoint: "0.0.0.0:8125"
    timer_histogram_mapping:
      - statsd_type: "histogram"
        observer_type: "histogram"
        histogram: {}
      - statsd_type: "timer"
        observer_type: "histogram"
        histogram: {}

processors:
  batch: {}

exporters:
  otlphttp:
    endpoint: "${DT_ENDPOINT}" # https://&lt;env-
id&gt;.live.dynatrace.com/api/v2/otlp
    headers:
      Authorization: "Api-Token ${DT_API_TOKEN}"

service:
  pipelines:
    metrics:
      receivers: [statsd]
      processors: [batch]
      exporters: [otlphttp]
```

Step 2: Deploy the collector in a dedicated namespace

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: otel-collector-statsd
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: otel-collector-statsd
  template:
    metadata:
      labels:
        app: otel-collector-statsd
    spec:
      containers:
        - name: otel-collector
          image: otel/opentelemetry-collector-contrib:0.151.0  # pinned — keep
in sync with OTEL-03/OTEL-07
          args: ["--config=/conf/otelcol-config.yaml"]
          ports:
            - containerPort: 8125
              protocol: UDP
          env:
            - name: DT_ENDPOINT
              valueFrom:
                secretKeyRef:
                  name: dynatrace-secret
                  key: endpoint
            - name: DT_API_TOKEN
              valueFrom:
                secretKeyRef:
                  name: dynatrace-secret
                  key: api-token
          volumeMounts:
            - name: config
              mountPath: /conf
      volumes:
        - name: config
          configMap:
            name: otel-collector-config
---
```

```
apiVersion: v1
kind: Service
metadata:
  name: otel-collector-statsd
  namespace: monitoring
spec:
  selector:
    app: otel-collector-statsd
  ports:
    - name: statsd-udp
      protocol: UDP
      port: 8125
      targetPort: 8125
```

Step 3: Point StatsD clients at the collector using the FQDN

Since the collector lives in the `monitoring` namespace, apps in other namespaces must use the fully qualified service name:

```
# In your application deployment (any namespace)
env:
  - name: STATSD_HOST
    value: "otel-collector-statsd.monitoring.svc.cluster.local"
  - name: STATSD_PORT
    value: "8125"
```

> **Note:** No application code changes are needed — just the env var configuration. The collector is managed independently from your application lifecycle.

Ownership Model

Component	Owner	Namespace
OTel Collector Deployment	Platform / Infra team	`monitoring`
Collector ConfigMap + Secret	Platform / Infra team	`monitoring`
Application `STATSD_HOST` env var	App team	App namespace

Requirements

Requirement	Details
API Token	`metrics.ingest` scope

> **Sources:** StatsD ingestion (DT docs) — *"For Kubernetes environments, we recommend remote StatsD monitoring using an environment ActiveGate."*
> | **Environment URL** | `https://<env-id>.live.dynatrace.com` |
> | **Collector Image** | `otel/opentelemetry-collector-contrib` (includes StatsD receiver) |
>
> **Tip:** You can also use the Dynatrace Collector image or build a custom collector with the OpenTelemetry Builder. The key requirement is the StatsD receiver component.

Verifying StatsD Metrics in Dynatrace

After deploying the collector and pointing StatsD clients at it, verify metrics are flowing:

```
// Verify StatsD metrics are being ingested
// The StatsD receiver keeps the metric names your client sends, so look them up by your own
// prefix. metrics takes from: with no leading comma.
metrics from:-1h
| filter startsWith(metric.key, "my_app.")
| summarize n = count(), by:{metric.key}
| sort metric.key asc
```

7. Troubleshooting Specialized Scenarios

NGINX Ingress Issues

Issue	Cause	Solution
Module load fails	Wrong architecture	ARM64 is not supported

Issue	Cause	Solution
No spans from ingress	Pod name mismatch	Pod or container name must contain `ingress-nginx-` or `nginx-ingress-`
Partial traces	OneAgent version	Upgrade to 1.227+

```
# Debug NGINX module loading
kubectl -n ingress-nginx exec -it deploy/ingress-nginx-controller -- \
  ls -la /opt/dynatrace/oneagent-paas/agent/bin/current/
```

CSI Driver Issues

Issue	Cause	Solution
Volume mount fails	Provisioner OOM	Increase provisioner limits
Slow pod startup	CSI driver overloaded	Add resources, check node count
Code modules missing	No delivery path working	Check the CSI pod on that node, or the image-volume / ephemeral-volume prerequisites (§ 2)

```
# Check CSI driver status
kubectl -n dynatrace get pods -l app.kubernetes.io/component=csi-driver

# View CSI driver logs
kubectl -n dynatrace logs -l app.kubernetes.io/component=csi-driver -c server
```

StatsD Ingestion Issues

Issue	Cause	Solution
No metrics arriving	Wrong collector image	Use `otel/opentelemetry-collector-contrib` (not base)
UDP traffic not reaching collector	Missing Service or wrong protocol	Ensure Service specifies `protocol: UDP`

Issue	Cause	Solution
Metrics visible but unnamed	Missing metric prefix	Configure StatsD client to include application prefix
Token errors in collector logs	Wrong scope	Token needs `metrics.ingest` scope

```
# Check OTel collector logs for StatsD errors
kubectl logs -l app=otel-collector-statsd --tail=50

# Test UDP connectivity to the collector
kubectl run statsd-test --rm -it --image=busybox -- \
  sh -c 'echo "test.metric:1|c" | nc -u -w1 otel-collector-statsd 8125'
```

Resource Exhaustion

Symptom	Component	Action
OOMKilled	Any	Increase memory limits
CPU throttling	Any	Increase CPU limits
Slow queries	ActiveGate	Add replicas or resources
Dropped spans	OTel Collector	Increase batch size, resources

Summary

In this notebook, you learned:

- **NGINX Ingress monitoring** with OneAgent module loading
- **Code-module delivery**: image volumes (Operator 1.11.0+, now recommended), the CSI driver and ephemeral volumes
- **CSI Driver resource configuration** with per-container limits
- **Component sizing guidelines** for all Dynatrace components
- **Telemetry ingest configuration** for multi-protocol support
- **StatsD ingestion on Kubernetes** via an environment ActiveGate, the DynaKube telemetry ingest endpoint, or a self-managed OpenTelemetry Collector
- **Troubleshooting** common specialized monitoring issues

References

- NGINX instrumentation (DT docs)
- Storage / CSI driver (DT docs)
- Dynatrace Operator Helm chart (Dynatrace GitHub)
- Telemetry ingest with OpenTelemetry (DT docs)
- StatsD ingestion (DT docs)
- StatsD via OpenTelemetry Collector (DT docs)
- Set up Dynatrace on Kubernetes (DT docs)
- Migrate from CSI driver to ephemeral volumes (DT docs)
- Operator 1.11.0 release notes (DT docs)
- Enable Dynatrace telemetry ingest endpoints (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.